

CHAPITRE 2

Faire réfléchir le programme

Objectif : comprendre comment un programme décide et répète une action.



Durée : 5 heures

M. Haithem BERKANE

Objectifs du chapitre

À la fin de ce chapitre, tu seras capable de :

- ✓ Comprendre ce qu'est un algorithme et une condition
- ✓ Écrire une condition avec if, else et elif
- ✓ Utiliser les opérateurs de comparaison (==, !=, <, >, <=, >=)
- ✓ Répéter une action un nombre de fois précis avec for et range()
- ✓ Répéter une action tant qu'une condition est vraie avec while
- ✓ Utiliser un compteur et un accumulateur dans une boucle

Ce qu'on réutilise du Chapitre 1



Une variable range une information : age = 16, prenom = "Ali"...



print() affiche quelque chose à l'écran ; input() demande une information au clavier.



int() et float() convertissent un texte en nombre pour pouvoir calculer.



Aujourd'hui, on va apprendre à faire choisir et répéter des actions au programme.

Un algorithme qui prend une décision



Un algorithme

Un algorithme est une suite d'étapes précises qui permet de résoudre un problème, même avant d'écrire la moindre ligne de code. On peut le décrire simplement, avec des mots.

Exemple : Pour savoir si on peut regarder un film : 1. Demander l'âge. 2. Si l'âge est suffisant, autoriser. 3. Sinon, refuser.



Une condition

Une condition est une question dont la réponse est soit vraie, soit fausse. Selon la réponse, le programme n'exécutera pas les mêmes instructions.

Exemple : « As-tu au moins 16 ans ? » est une condition : la réponse est forcément vraie (True) ou fausse (False).

La comparaison : vrai ou faux



```
1 age = 16
2 print(age >= 18)
```

🔍 Avant de lire la suite : Python va-t-il répondre True ou False ?

age >= 18 se lit « est-ce que age est supérieur ou égal à 18 ? ». Il existe aussi <, >, <=, == et !=. Python répond toujours par True ou False.



CE QUE PYTHON FAIT, ÉTAPE PAR ÉTAPE

`age = 16`

→ Python range 16 dans la boîte age

`age >= 18 ?`

→ 16 n'est pas $\geq$ 18

`résultat`

→ affiche False

if : exécuter une action seulement si...



```
1 age = 20
2 if age >= 18:
3     print("Majeur")
```

? Que va afficher ce programme, d'après toi ?



CE QUE PYTHON FAIT, ÉTAPE PAR ÉTAPE

`age = 20`

→ Python range 20 dans age

`age >= 18 ?`

→ $20 \geq 18 \rightarrow \text{True}$

`condition vraie`

→ le bloc indenté s'exécute →
affiche Majeur

Après if condition:, il faut TOUJOURS deux-points. La ligne indentée juste après n'est exécutée QUE si la condition est vraie.

if : et si la condition est fausse ?



```
1 age = 15
2 if age >= 18:
3     print("Majeur")
```

? Cette fois, age vaut 15. Que va-t-il se passer ?



CE QUE PYTHON FAIT, ÉTAPE PAR ÉTAPE

`age = 15`

→ Python range 15 dans age

`age >= 18 ?`

→ 15 n'est pas $\geq$ 18 → False

`condition fausse`

→ le bloc indenté est ignoré → rien ne s'affiche

C'est une idée essentielle à retenir : si la condition est fausse, Python saute complètement le bloc indenté et continue après, sans erreur ni message.

if : je m'entraîne

 AVEC TOI

Complète le code pour afficher « Réussi » seulement si note est supérieure ou égale à 10.

```
note = 12

if note ____ 10:

    print("Réussi")
```

 **À compléter :** Remplace ____ par l'opérateur de comparaison « supérieur ou égal à ». Avant d'exécuter, simule dans ta tête : note vaut 12, est-ce que 12 respecte la condition ?

 SEUL

Crée une variable temperature = 30 et affiche « Il fait chaud » seulement si temperature est supérieure à 25. Simule d'abord mentalement, puis écris le code seul, sans regarder l'exemple précédent.

 **Indice :** N'oublie pas les deux-points à la fin de la ligne if, et l'indentation de la ligne qui suit.

else et elif : gérer plusieurs cas



```
1 age = 15
2 if age >= 18:
3     print("Majeur")
4 elif age >= 13:
5     print("Adolescent")
6 else:
7     print("Enfant")
```

? Simule ligne par ligne : quelle condition va être vraie en premier ?

CE QUE PYTHON FAIT, ÉTAPE PAR ÉTAPE

`age = 15` → Python range 15 dans age

`age >= 18 ?` → $15 \geq 18 \rightarrow \text{False}$, on passe à elif

`age >= 13 ?` → $15 \geq 13 \rightarrow \text{True}$

résultat → affiche Adolescent, et s'arrête là (else ignoré)

elif signifie « sinon si » : il n'est testé que si le if précédent est faux. else s'exécute seulement si RIEN n'était vrai avant. Jamais de condition après else.

Erreur fréquente : confondre = et ==

✗ Ce code provoque une erreur

```
age = 16  
if age = 18:  
    print("OK")
```

? Simule : à quoi sert le signe = ici ? Et si on voulait comparer ?

🔍 Pourquoi ça bloque :

- = sert à ranger une valeur dans une variable (une affectation).
- == sert à comparer deux valeurs (un test d'égalité).
- Dans un if, on veut TESTER une égalité : il faut donc obligatoirement ==.

✓ Version corrigée

```
age = 16  
if age == 18:  
    print("OK")
```

if / elif / else : je m'entraîne

🟡 AVEC TOI

Complète le code pour afficher l'appréciation selon la note : « Très bien » si ≥ 16 , sinon « Bien » si ≥ 12 , sinon « Insuffisant ». Simule d'abord avec `note = 14` avant de compléter.

```
note = 14

if note >= 16:

    print("Très bien")

__ note >= 12:

    print("Bien")

else:

    print("Insuffisant")
```

🟢 SEUL

Écris seul un programme qui affiche « Chaud », « Doux » ou « Froid » selon une variable `temperature` (≥ 25 : Chaud, ≥ 10 : Doux, sinon : Froid). Simule mentalement avec `temperature = 18` avant d'écrire ton code.

💡 **Indice** : Utilise `if` pour le premier cas, `elif` pour le(s) suivant(s), et `else` pour tout le reste.

for et range() : répéter un nombre de fois précis



```
1 for i in range(3):  
2     print(i)
```

? Avant de regarder la colonne de droite : que va afficher ce programme ?



CE QUE PYTHON FAIT, ÉTAPE PAR ÉTAPE

i = 0 → affiche 0

i = 1 → affiche 1

i = 2 → affiche 2

range(3) génère les nombres 0, 1, 2 (jamais le 3). La boucle s'arrête automatiquement après le dernier tour, sans instruction supplémentaire.

for et range() : je m'entraîne

🟡 AVEC TOI

Complète le code pour afficher les nombres de 0 à 4 (cinq nombres au total). Simule d'abord : `range(3)` donnait 0,1,2 — que donnera `range(5)` ?

```
for i in range(____):  
    print(i)
```

✏️ **À compléter :** Remplace ____ par le nombre qui permet d'obtenir 0, 1, 2, 3 et 4 (rappelle-toi : `range(n)` s'arrête avant `n`).

🟢 SEUL

Écris seul une boucle `for` qui affiche les 5 premiers multiples de 3 (3, 6, 9, 12, 15) en calculant `3 * i` à chaque tour. Simule d'abord les 2 premiers tours dans ta tête.

💡 **Indice :** `range(5)` donne les valeurs 0, 1, 2, 3, 4 : utilise-les pour calculer `3 * i` à chaque tour de boucle.

while : répéter tant qu'une condition est vraie



```
1  compteur = 0
2  while compteur < 3:
3      print(compteur)
4      compteur = compteur + 1
```

? Simule tour par tour : à quel moment la condition devient-elle fausse ?



CE QUE PYTHON FAIT, ÉTAPE PAR ÉTAPE

`compteur=0` → `0<3` vrai → affiche 0

`compteur=1` → `1<3` vrai → affiche 1

`compteur=2` → `2<3` vrai → affiche 2

`compteur=3` → `3<3` faux → arrêt

compteur = compteur + 1 augmente la variable de 1 à chaque tour : on appelle cela un compteur. Sans cette ligne, la condition resterait toujours vraie.

Erreur fréquente : la boucle infinie

✗ Ce code provoque une erreur

```
compteur = 0
while compteur < 3:
    print(compteur)
```

? Pourquoi ce programme ne s'arrête-t-il jamais ?

🔍 Pourquoi ça bloque :

- compteur ne change jamais de valeur à l'intérieur de la boucle.
- La condition `compteur < 3` reste donc toujours vraie, pour toujours.
- Une boucle `while` a besoin d'une instruction qui finit par rendre sa condition fausse.

✓ Version corrigée

```
compteur = 0
while compteur < 3:
    print(compteur)
    compteur = compteur + 1
```

while : je m'entraîne

 AVEC TOI

Complète le code pour afficher les nombres de 0 à 4 avec une boucle while.

```
i = 0  
while i < ____:  
    print(i)  
    i = i + 1
```

 **À compléter :** Remplace ____ par le nombre qui arrête la boucle juste après avoir affiché 4.

 SEUL

Écris une boucle while qui calcule la somme des nombres de 1 à 5, à l'aide d'un accumulateur somme.

 **Indice :** Un accumulateur est une variable qui grandit à chaque tour de boucle : `somme = somme + i`.

Ce qu'il faut retenir du Chapitre 2



Une condition est une question vraie ou fausse (True / False).



if exécute un bloc seulement si la condition est vraie.



elif teste une autre condition ; else s'exécute sinon.



for ... in range(n) répète un bloc n fois, de 0 à n-1.



while répète tant qu'une condition reste vraie.



Un compteur ou un accumulateur doit être mis à jour à chaque tour.

N°1

CAS PRATIQUE — Tester une condition (majorité)

Mémoriser

Niveau 1-2

Écrire une condition simple avec if.

 **Situation** : Tu veux vérifier si quelqu'un est majeur (18 ans ou plus).

 **Problème** : Comment demander à Python de tester si age est supérieur ou égal à 18 ?

 **Consigne** : Crée age = 20, puis affiche « Majeur » si age >= 18.

 **Indice 1** : Utilise if suivi de la condition, puis des deux-points :.

 **Indice 2** : N'oublie pas d'indenter (décaler) la ligne print().

 **À toi** : refais ce raisonnement seul, avec d'autres valeurs.

 **Simulation : on exécute mentalement**

```
→ age = 20
→ 20 ≥ 18 → True
→ Python exécute le bloc → affiche Majeur
```

 **Solution**

```
age = 20
if age >= 18:
    print("Majeur")
```

 **À retenir** : La ligne indentée ne s'exécute que si la condition juste au-dessus est vraie.

 **Erreur fréquente** : Oublier les deux-points à la fin de la ligne if, ou oublier d'indenter la ligne print().

Réappliquer if avec une nouvelle variable.

 **Situation** : Un élève veut savoir si sa note lui permet de valider (10 ou plus).

 **Problème** : Comment tester si note est supérieure ou égale à 10 ?

 **Consigne** : Crée note = 12, puis affiche « Validé » si note >= 10.

 **Indice 1** : La structure est identique à l'exemple précédent : if condition:

 **Indice 2** : Remplace simplement la variable et le seuil.

 **À toi** : refais ce raisonnement seul, avec d'autres valeurs.

 **Simulation : on exécute mentalement**

→ note = 12

→ 12 ≥ 10 → True

→ Python exécute le bloc → affiche Validé

 **Solution**

```
note = 12

if note >= 10:
    print("Validé")
```

 **À retenir** : Même mécanique que pour l'âge : seule la variable et le seuil changent.

 **Erreur fréquente** : Utiliser > au lieu de >= : une note de exactement 10 ne serait alors plus considérée comme validée.

N°3

CAS PRATIQUE — if / else : deux chemins possibles

Comprendre

Niveau 2-3

Utiliser else pour gérer le cas contraire.

 **Situation** : On veut afficher un message différent selon qu'un nombre est pair ou impair.

 **Problème** : Comment exécuter une instruction différente si la condition est fausse ?

 **Consigne** : Teste si `nombre % 2 == 0` (pair) ; sinon affiche qu'il est impair.

 **Indice 1** : `%` est l'opérateur modulo : il donne le reste d'une division.

 **Indice 2** : `else` ne prend jamais de condition après lui, juste des deux-points.

 **À toi** : refais ce raisonnement seul, avec d'autres valeurs.

 **Simulation : on exécute mentalement**

→ `nombre = 7`

→ `7 % 2 = 1`, donc `1 == 0` → `False`

→ le `if` est faux → Python exécute le bloc `else` → affiche `Impair`

 **Solution**

```
nombre = 7

if nombre % 2 == 0:
    print("Pair")
else:
    print("Impair")
```

 **À retenir** : `nombre % 2` donne le reste de la division par 2 : 0 pour un nombre pair, 1 pour un impair.

 **Erreur fréquente** : Oublier `==` et écrire `nombre % 2 = 0`, ce qui provoque une erreur de syntaxe.

M. Haithem BERKANE

N°4

CAS PRATIQUE — if / elif / else : plusieurs cas

Gérer plus de deux cas possibles.

Comprendre

Niveau 2-3

📌 **Situation** : Un système attribue une appréciation : Très bien (≥ 16), Bien (≥ 12), Insuffisant sinon.

? **Problème** : Comment gérer plus de deux cas possibles dans un seul programme ?

📋 **Consigne** : Écris un programme qui affiche l'appréciation correspondant à `note = 14`.

💡 **Indice 1** : Utilise `if` pour le premier cas, `elif` pour le(s) suivant(s), `else` pour le reste.

💡 **Indice 2** : Python teste les conditions dans l'ordre et s'arrête à la première qui est vraie.

✍️ **À toi** : refais ce raisonnement seul, avec d'autres valeurs.

🔍 **Simulation : on exécute mentalement**

```
→ note = 14
→ note >= 16 ? 14 ≥ 16 → False
→ note >= 12 ? 14 ≥ 12 → True → affiche Bien, et s'arrête là
```

💻 **Solution**

```
note = 14
if note >= 16:
    print("Très bien")
elif note >= 12:
    print("Bien")
else:
    print("Insuffisant")
```

🔍 **À retenir** : Python s'arrête à la première condition vraie qu'il rencontre, dans l'ordre d'écriture.

⚠️ **Erreur fréquente** : Utiliser plusieurs `if` indépendants au lieu de `elif` : plusieurs messages pourraient s'afficher à la fois.

N°5

CAS PRATIQUE — Compléter une boucle for

Compléter un code à trous utilisant `range()`.

Appliquer

Niveau 3-4

 **Situation** : On veut afficher les nombres de 0 à 4.

 **Problème** : Une partie de l'instruction `range()` est manquante.

 **Consigne** : Complète `range(____)` pour afficher exactement les nombres 0, 1, 2, 3, 4.

 **Indice 1** : `range(n)` génère les nombres de 0 à `n-1`, jamais `n` lui-même.

 **Indice 2** : Pour aller jusqu'à 4 inclus, il faut donc `range(5)`.

 **À toi** : refais ce raisonnement seul, avec d'autres valeurs.

 **Simulation : on exécute mentalement**

→ `range(5)` → 0, 1, 2, 3, 4

→ 5 valeurs générées, la dernière est 4 (jamais 5)

→ chaque valeur est affichée par `print(i)`

 **Solution**

```
for i in range(5):  
    print(i)
```

 **À retenir** : `range(5)` produit cinq valeurs (0 à 4) : le nombre entre parenthèses n'est jamais atteint.

 **Erreur fréquente** : Écrire `range(4)`, ce qui n'affiche que 0, 1, 2, 3 et oublie le nombre 4.

Identifier une confusion entre = et ==.

 **Situation** : Un élève a écrit ce code pour tester un mot de passe.

 **Problème** : `if mot_de_passe = "python":` — pourquoi Python affiche-t-il une erreur de syntaxe ?

 **Consigne** : Identifie l'erreur et corrige le code.

 **Indice 1** : `=` sert à donner une valeur, `==` sert à comparer deux valeurs.

 **Indice 2** : Dans un `if`, on a toujours besoin d'une comparaison, donc de `==`.

 **À toi** : refais ce raisonnement seul, avec d'autres valeurs.

 **Simulation : on exécute mentalement**

→ `Python lit if mot_de_passe = "python":`

→ `=` n'est pas un opérateur de comparaison valide dans un `if`

→ Python s'arrête immédiatement : `SyntaxError`, avant même d'exécuter quoi que ce soit

 **Solution**

```
mot_de_passe = "python"
if mot_de_passe == "python":
    print("Accès autorisé")
```

 **À retenir** : `=` range une valeur, `==` compare deux valeurs. Un `if` a toujours besoin d'une comparaison.

 **Erreur fréquente** : Corriger uniquement l'un des deux endroits et laisser `=` à la place de `==` ailleurs.

N°7

CAS PRATIQUE — Modifier une boucle existante

Adapter les bornes d'une boucle for.

Appliquer

Niveau 4

 **Situation** : Un programme affiche actuellement les nombres de 0 à 9.

 **Problème** : Comment le modifier pour qu'il affiche plutôt les nombres de 1 à 10 ?

 **Consigne** : Pars de `for i in range(10):` et adapte le code pour obtenir 1 à 10.

 **Indice 1** : `range(1, 11)` démarre à 1 et s'arrête juste avant 11.

 **Indice 2** : `range()` peut recevoir deux valeurs : un début et une fin (exclue).

 **À toi** : refais ce raisonnement seul, avec d'autres valeurs.

 **Simulation : on exécute mentalement**

→ `range(1, 11)` → 1, 2, 3 ... 10

→ premier tour : `i = 1` (pas 0 cette fois)

→ dernier tour : `i = 10`, puis arrêt avant 11

 **Solution**

```
for i in range(1, 11):  
    print(i)
```

 **À retenir** : `range(debut, fin)` commence à `debut` et s'arrête juste avant `fin`.

 **Erreur fréquente** : Écrire `range(1, 10)`, qui s'arrête à 9 et oublie le nombre 10.

Résoudre seul un problème utilisant une boucle for.

 **Situation** : Tu veux afficher la table de multiplication de 5, de 1 à 10.

 **Problème** : Comment répéter le calcul $5 * i$ pour chaque valeur de i entre 1 et 10 ?

 **Consigne** : Écris une boucle for qui affiche $5 * i$ pour i allant de 1 à 10.

 **Indice 1** : Utilise `range(1, 11)` pour aller de 1 à 10 inclus.

 **Indice 2** : À l'intérieur de la boucle, calcule `resultat = 5 * i` puis affiche-le.

 **À toi** : refais ce raisonnement seul, avec d'autres valeurs.

 **Simulation : on exécute mentalement**

→ tour 1 : $i=1 \rightarrow \text{resultat} = 5*1 = 5$
→ tour 2 : $i=2 \rightarrow \text{resultat} = 5*2 = 10$
→ ... et ainsi de suite jusqu'à $i=10 \rightarrow 50$

 **Solution**

```
for i in range(1, 11):  
    resultat = 5 * i  
    print(resultat)
```

 **À retenir** : À chaque tour, i change de valeur et le calcul $5 * i$ est refait avec cette nouvelle valeur.

 **Erreur fréquente** : Placer `resultat = 5 * i` en dehors de la boucle (mal indenté) : le calcul ne serait fait qu'une fois.

Combiner une boucle et une condition avec un accumulateur.

 **Situation** : On veut savoir combien de nombres pairs se trouvent entre 1 et 20.

 **Problème** : Comment combiner une boucle et une condition pour compter les nombres pairs ?

 **Consigne** : Utilise un for et un if pour compter les nombres pairs de 1 à 20, avec un accumulateur compteur.

 **Indice 1** : Initialise compteur = 0 avant la boucle.

 **Indice 2** : À chaque nombre pair trouvé, fais compteur = compteur + 1.

 **À toi** : refais ce raisonnement seul, avec d'autres valeurs.

 **Simulation : on exécute mentalement**

```
→ i=1 impair → compteur=0
→ i=2 pair → compteur=1
→ ... jusqu'à i=20 → compteur final=10
```

 **Solution**

```
compteur = 0

for i in range(1, 21):
    if i % 2 == 0:
        compteur = compteur + 1

print(compteur)
```

 **À retenir** : compteur n'augmente que lorsque la condition (nombre pair) est vraie : il compte au fil de la boucle.

 **Erreur fréquente** : Placer compteur = 0 dans la boucle : il serait réinitialisé à chaque tour, et resterait à 0 ou 1.

N°10

CAS PRATIQUE — Défi final du chapitre

Mobiliser boucle, condition et accumulateurs dans un même programme.

Créer

Niveau 6

 **Situation** : Tu veux créer un petit programme qui teste plusieurs nombres saisis et compte ceux au-dessus d'une limite.

 **Problème** : Comment combiner une boucle while, une condition et un accumulateur ?

 **Consigne** : Demande 5 nombres à l'utilisateur (boucle while + compteur de tours), et affiche combien sont supérieurs à 10.

 **Indice 1** : Utilise un compteur de tours (ex : tour = 0) et une condition while tour < 5.

 **Indice 2** : Utilise un second accumulateur (ex : resultat = 0) pour compter les nombres supérieurs à 10.

 **À toi** : refais ce raisonnement seul, avec d'autres valeurs.

Simulation : on exécute mentalement

→ tour=0, saisie 15 → resultat=1
→ tour=1, saisie 4 → resultat inchangé
→ ... jusqu'à tour=5 → arrêt, on affiche resultat

Solution

```
tour = 0
resultat = 0
while tour < 5:
    nombre = int(input("Un nombre : "))
    if nombre > 10:
        resultat = resultat + 1
    tour = tour + 1
print(resultat)
```

 **À retenir** : tour compte les répétitions (compteur), resultat compte les nombres qui respectent la condition (accumulateur).

 **Erreur fréquente** : Oublier tour = tour + 1 : la boucle while ne s'arrêterait alors jamais (boucle infinie).

N°11

CAS PRATIQUE — Comparer deux nombres

Comprendre

Niveau 2-3

Gérer trois cas possibles avec `if / elif / else`.

 **Situation** : Tu veux savoir lequel de deux nombres est le plus grand, ou s'ils sont égaux.

 **Problème** : Comment gérer les trois cas possibles (a plus grand, b plus grand, égaux) ?

 **Consigne** : Compare `a = 8` et `b = 8` et affiche le bon message parmi les trois cas.

 **Indice 1** : Utilise `if`, `elif`, puis `else` pour couvrir les trois possibilités.

 **Indice 2** : Le cas d'égalité se teste avec `==`.

 **À toi** : refais ce raisonnement seul, avec d'autres valeurs.

Simulation : on exécute mentalement

→ `a=8, b=8`

→ `a > b ? 8 > 8 → False. a < b ? 8 < 8 → False`

→ aucune des deux n'est vraie → Python exécute `else`

→ affiche « égaux »

Solution

```
a = 8
b = 8
if a > b:
    print("a est plus grand")
elif a < b:
    print("b est plus grand")
else:
    print("Les deux sont égaux")
```

 **À retenir** : `else` couvre automatiquement tous les cas restants, ici l'égalité, sans avoir besoin de la tester explicitement.

 **Erreur fréquente** : Ne tester que `if a > b / else` sans `elif` : le cas d'égalité serait mal géré.

N°12

CAS PRATIQUE — Trouver l'erreur : boucle qui ne s'arrête jamais

Analyser

Niveau 3-4

Repérer l'absence d'incrémentation dans une boucle while.

 **Situation** : Un élève a écrit une boucle qui doit afficher 0, 1 et 2, mais le programme ne s'arrête plus.

 **Problème** : `i = 0 / while i < 3: / print(i)` — pourquoi cette boucle est-elle infinie ?

 **Consigne** : Identifie ce qui manque dans la boucle et corrige le code.

 **Indice 1** : Compare avec les exemples vus : la variable `i` doit-elle changer quelque part ?

 **Indice 2** : Il manque une ligne qui augmente `i` à chaque tour.

 **À toi** : refais ce raisonnement seul, avec d'autres valeurs.

Simulation : on exécute mentalement

→ `i=0` : `0<3` vrai → affiche 0, `i` reste 0

→ `i=0` (encore) : `0<3` vrai → affiche 0, `i` reste 0

→ ... ce tour se répète pour toujours : la boucle ne s'arrête jamais

Solution

```
i = 0

while i < 3:
    print(i)
    i = i + 1
```

 **À retenir** : Sans `i = i + 1`, la variable ne change jamais : la condition reste vraie indéfiniment.

 **Erreur fréquente** : Ajouter `i = i + 1` mais mal l'indenter, en dehors du bloc `while` : cela ne résout pas le problème.

M. Haithem BERKANE

N°13

CAS PRATIQUE — Modifier un programme : du for au while

Appliquer

Niveau 4

Réécrire une boucle for en boucle while équivalente.

 **Situation** : Ce programme affiche les nombres de 0 à 4 avec un for.

 **Problème** : Comment obtenir exactement le même résultat avec une boucle while ?

 **Consigne** : Réécris `for i in range(5): print(i)` en utilisant while.

 **Indice 1** : Il faut créer une variable `i = 0` avant la boucle.

 **Indice 2** : N'oublie pas d'incrémenter `i` à chaque tour, sinon la boucle est infinie.

 **À toi** : refais ce raisonnement seul, avec d'autres valeurs.

 **Simulation : on exécute mentalement**

→ `i=0` : `0<5` vrai → affiche 0, puis `i=1`

→ `i=1` : `1<5` vrai → affiche 1, puis `i=2`

→ ... jusqu'à `i=5` : `5<5` faux → arrêt (mêmes valeurs que le for)

 **Solution**

```
i = 0
while i < 5:
    print(i)
    i = i + 1
```

 **À retenir** : for gère automatiquement ce que while doit faire à la main : initialiser, tester, incrémenter.

 **Erreur fréquente** : Utiliser `while i <= 5` au lieu de `while i < 5` : cela afficherait un nombre de trop (5).

N°14

CAS PRATIQUE — Petit problème autonome : somme des nombres pairs

Appliquer

Niveau 5

Combiner boucle, condition et accumulateur.

 **Situation** : Tu veux calculer la somme de tous les nombres pairs entre 1 et 10.

 **Problème** : Comment combiner une boucle, une condition et un accumulateur pour cela ?

 **Consigne** : Utilise une boucle et une condition pour additionner uniquement les nombres pairs de 1 à 10.

 **Indice 1** : Initialise somme = 0 avant la boucle.

 **Indice 2** : N'ajoute à somme que lorsque $i \% 2 == 0$.

 **À toi** : refais ce raisonnement seul, avec d'autres valeurs.

 **Simulation : on exécute mentalement**

→ $i=2$ (pair) → somme = $0+2 = 2$

→ $i=4$ (pair) → somme = $2+4 = 6$

→ ... $i=6,8,10$ → somme finale = $2+4+6+8+10 = 30$

 **Solution**

```
somme = 0

for i in range(1, 11):
    if i % 2 == 0:
        somme = somme + i

print(somme)
```

 **À retenir** : somme n'est mise à jour que lorsque la condition (nombre pair) est vraie.

 **Erreur fréquente** : Écrire `somme = somme + i` hors du `if` : tous les nombres seraient additionnés, pas juste les pairs.

Utiliser une boucle et une variable pour trouver un maximum.

 **Situation** : Tu veux demander 3 nombres à l'utilisateur et afficher le plus grand des trois.

 **Problème** : Comment garder en mémoire le plus grand nombre vu jusqu'ici, au fil des saisies ?

 **Consigne** : Utilise une boucle et une variable `plus_grand` pour trouver le maximum parmi 3 nombres saisis.

 **Indice 1** : Initialise `plus_grand` avec une valeur très petite, ou avec le premier nombre saisi.

 **Indice 2** : À chaque tour, compare le nouveau nombre à `plus_grand` et mets-le à jour si besoin.

 **À toi** : refais ce raisonnement seul, avec d'autres valeurs.

Simulation : on exécute mentalement

→ saisie 7 : `7 > 0` vrai → `plus_grand` devient 7
→ saisie 3 : `3 > 7` faux → `plus_grand` reste 7
→ saisie 12 : `12 > 7` vrai → `plus_grand` devient 12

Solution

```
plus_grand = 0
for i in range(3):
    nombre = int(input("Un nombre : "))
    if nombre > plus_grand:
        plus_grand = nombre
print(plus_grand)
```

 **À retenir** : `plus_grand` est mise à jour uniquement quand une valeur plus grande est trouvée : à la fin, elle contient le maximum.

 **Erreur fréquente** : Initialiser `plus_grand` à 0 pose problème si tous les nombres sont négatifs : mieux vaut partir du premier saisi.

Mini-évaluation : Je sais faire

Coche chaque compétence quand tu te sens capable de la réaliser seul(e).

☐

Je sais écrire une condition avec if.

☐

Je sais utiliser else pour gérer le cas contraire.

☐

Je sais utiliser elif pour gérer plusieurs cas.

☐

Je sais écrire une boucle for avec range().

☐

Je sais écrire une boucle while.

☐

Je sais utiliser un compteur ou un accumulateur dans une boucle.

Mon petit détecteur de nombres

Écris un programme qui demande 5 nombres à l'utilisateur (boucle), et qui affiche à la fin combien d'entre eux sont pairs et combien sont impairs. Réutilise for ou while, if/else, et deux accumulateurs.

CE QUE TU SAIS MAINTENANT FAIRE

-  Écrire une condition avec if
-  Gérer un cas alternatif avec else
-  Gérer plusieurs cas avec elif
-  Répéter un bloc avec for et range()
-  Répéter un bloc avec while
-  Utiliser un compteur ou un accumulateur